



Vijay Kumar

JavaScript Training

PRESENTATION BY:
VIJAY KUMAR



Asha Learnology

What is JavaScript?

1. **High-level, interpreted programming language:** This means code is written in a way that's easier for humans to understand and doesn't require manual translation into machine code.
2. **Primarily for creating dynamic web content:** JavaScript adds interactivity to web pages, allowing elements to update, respond to user actions, and create a more engaging experience.
3. **Increasingly used in mobile development frameworks like React Native:** React Native lets you build native mobile apps using JavaScript, enabling developers to write code once and deploy it to both Android and iOS platforms.
4. **Client-side vs. Server-side:** Traditionally, JavaScript executes on the user's device (client-side) in a web browser. However, JavaScript engines now run on servers (server-side) as well, allowing for more complex interactions and data processing.
5. **Single-threaded:** JavaScript can only execute one task at a time, but techniques like asynchronous programming help manage multiple operations efficiently.



Variables: The Data Containers

Declaring Variables:

To create a variable, use the var, let, or const keyword followed by the chosen name

```
var username = "Vijay";
```

```
let age = 25;
```

```
const PI = 3.14159; // Use const for fixed values
```

Accessing Variables.

```
console.log("Hello, " + username);  
// Outputs: Hello, Vijay
```

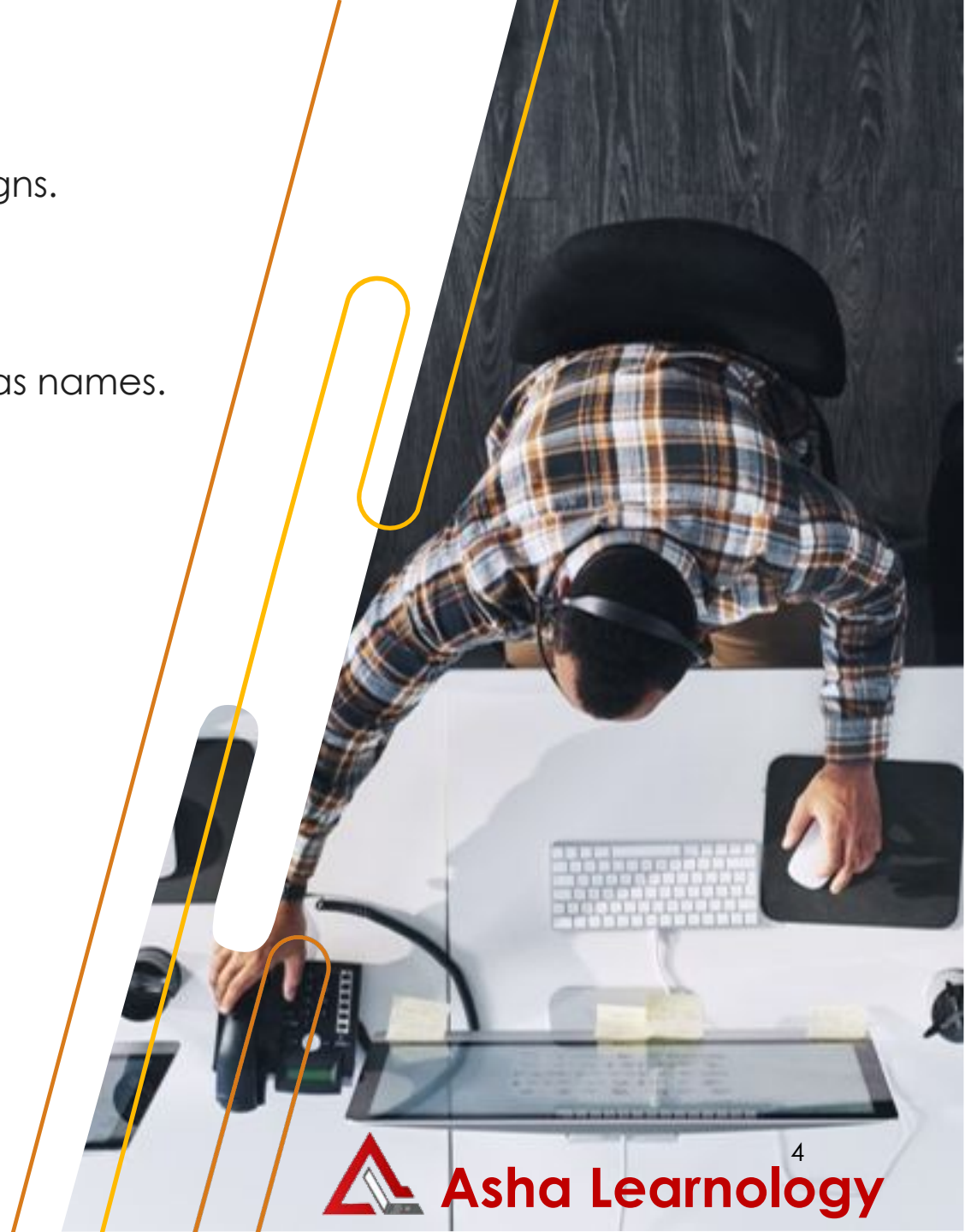
```
console.log("Your age is: " + age);  
// Outputs: Your age is: 25
```


Variables Naming Conventions:

1. Names can contain letters, digits, underscores, and dollar signs.
2. Names should begin with a letter, \$ and _ not number.
3. Names are case sensitive (y and Y are different variables).
4. Reserved words (like JavaScript keywords) cannot be used as names.

When to Use var, let, or const?

1. Always declare variables
2. Always use const if the value should not be changed
3. Always use const if the type should not be changed (Arrays and Objects)
4. Only use let if you can't use const
5. Only use var if you MUST support old browsers.



Data Type in Javascript

Primitive and Non Primitive

Number

To store numbers
(64-bit floating point) .
e.g. const age = 50;

Big int

To store any word.
e.g., const val=
54848464646464864488n ;

string

To store true / false only.
e.g. const name = "Vijay";

boolean

To store true / false only.
e.g. const isRunning = true;

undefined

When we have declared
variable but not defined
any value

null

When we are not sure about type of
value we can store null as value. e.g.
const data = null;

Some Other Data Types and typeof operator

Array, Objects, Sets, Map

Operators in Javascript

- **Arithmetic Operators:**

addition (+), subtraction (-), multiplication (*), division (/), and remainder (modulus %)

e.g. `let sum = 10 + 20; // sum will be 30`

- **Assignment Operators:** =, +=, -=, *=, /=, % =

e.g. `let count = 5; count += 3; // count will be 8`

- **Comparison Operators:**

== (equal to), === (exactly equal to), != (not equal to), !== (exactly not equal to), < (less than), > (greater than), <= (less than or equal to), and >= (greater than or equal to). e.g. `if (age >= 18) { ... } // Checks if age is 18 or above`

- **Logical Operators:** && (AND), || (OR), and ! (NOT).

e.g. `if (username && password) { ... } // Checks if both username and password are provided.`

- **Conditional (Ternary) Operator:** Shorthand way of if-else

e.g. `let message = (age >= 18) ? "Welcome!" : "Access denied";`

- **Unary Operators:** increment (++), decrement (--), and logical NOT (!)

e.g. `let count = 1; count++; // count will become 2.`

`let isRun = !isWalk; (set boolean isRun to Opposite of isWalk ).`

Conditions in Javascript

- **Using if**

```
let grade = "A";  
if (grade === "A") {  
  console.log("Excellent work!");  
}
```

- **Using if else**

```
let grade = "A";  
if (grade === "A") {  
  console.log("Excellent work!");  
} else {  
  console.log("Keep practicing!");  
}
```

- Using if – else if – else (else if ladder)

```
let score = 85;
if (score >= 90) {
  console.log("Outstanding!");
} else if (score >= 80) {
  console.log("Great job!");
} else {
  console.log("Don't give up!");
}
```

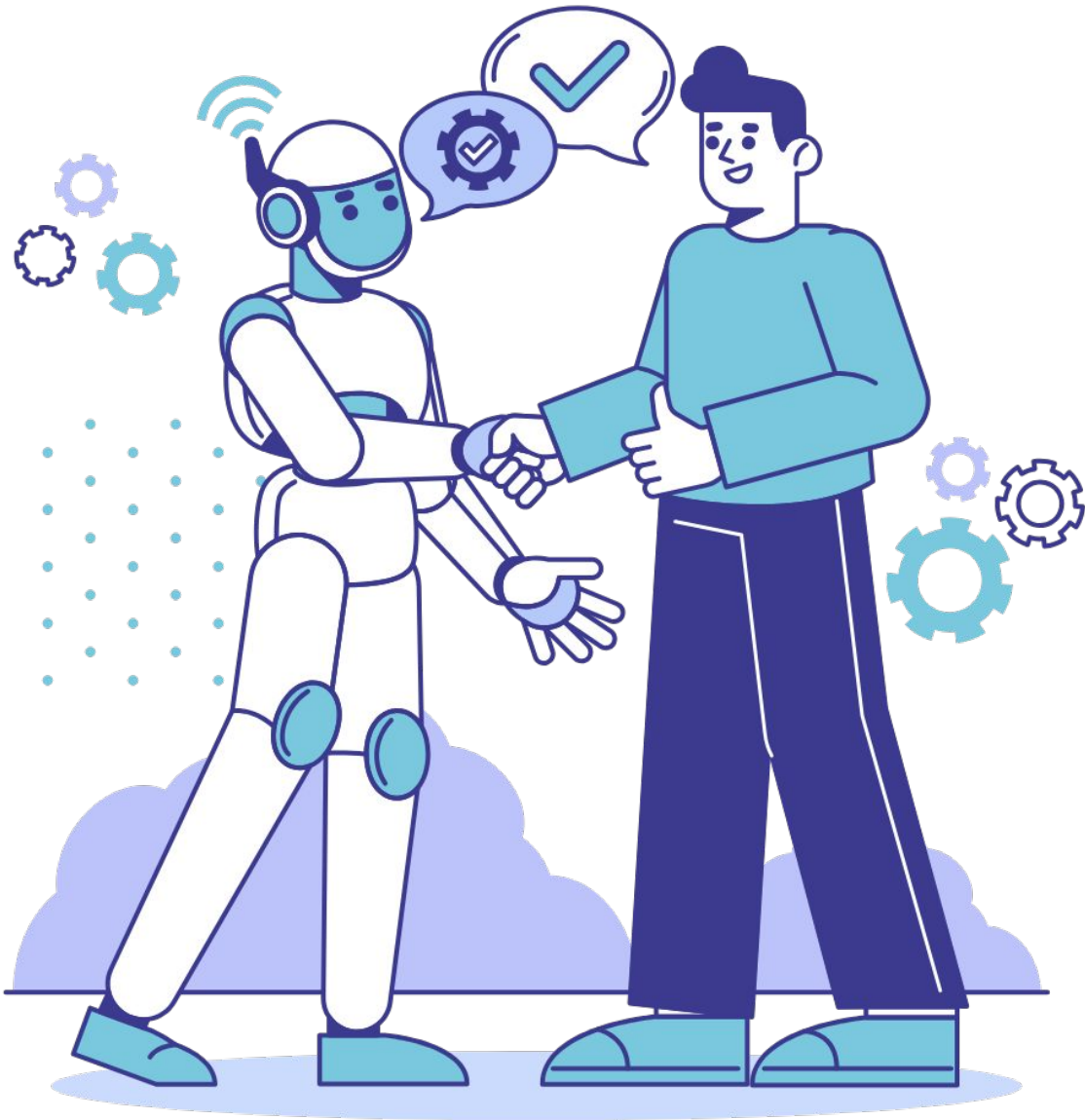
```
let score = 85;
if (score >= 90) {
  console.log("Outstanding!");
} else if (score >= 80) {
  console.log("Great job!");
} else if (score >= 70) {
  console.log("Good job!");
} else if (score >= 60) {
  console.log("fair job!");
} else {
  console.log("Don't give up!");
}
```


- Using Nested if else

```
let user = "be1newinner";  
let pwrđ = "123";  
if (user === "be1newinner") {  
  if(pwrđ === "123"){  
    Console.log("accessed!")  
  } else {  
    Console.log("invalid pwrđ!")  
  }  
} else {  
  console.log("Invalid user!");  
}
```



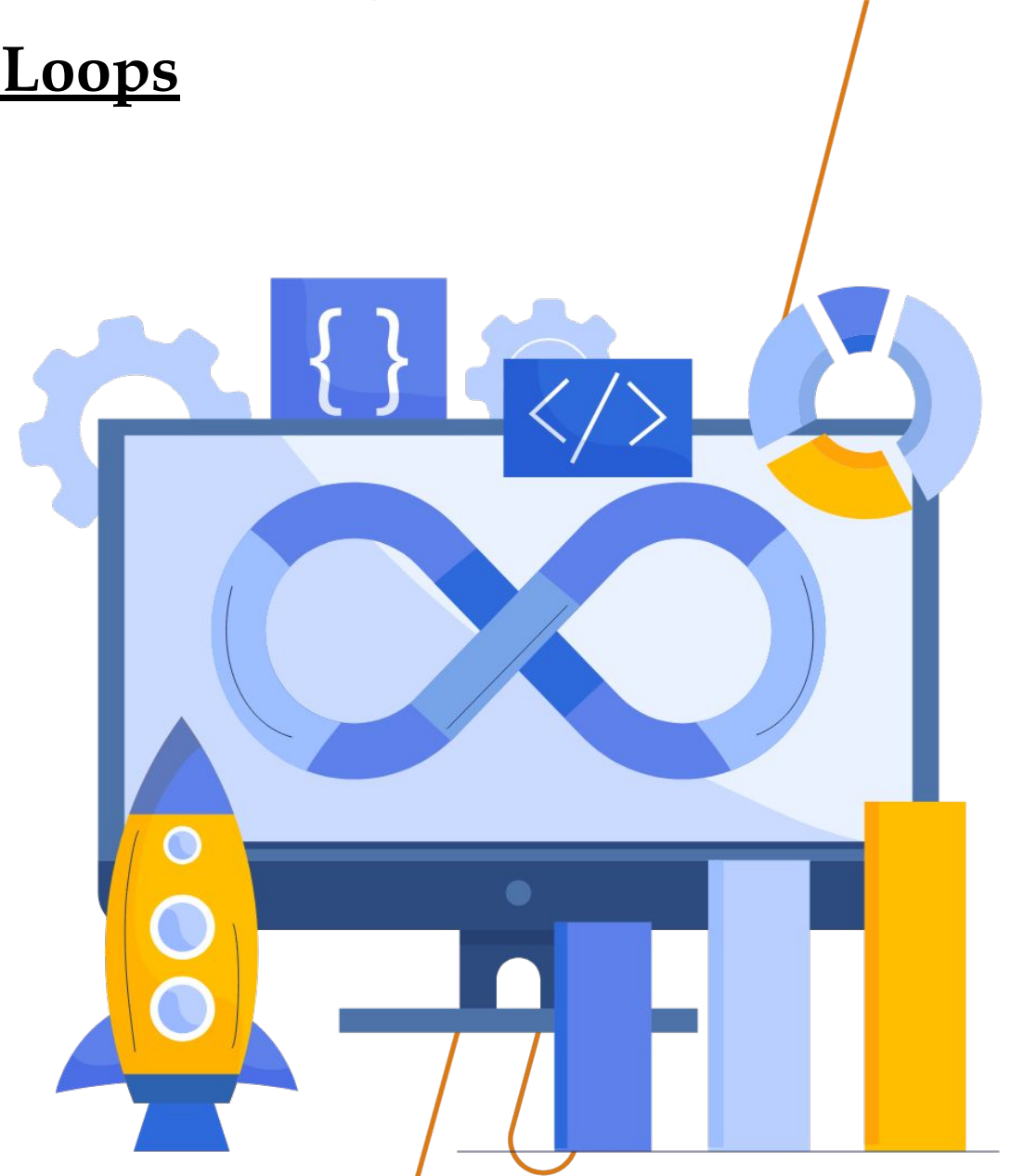
Switch in Javascript



```
let day = "Tuesday";  
switch (day) {  
  case "Monday":  
  case "Tuesday":  
  case "Wednesday":  
    console.log("It's a weekday.");  
    break;  
  case "Saturday":  
  case "Sunday":  
    console.log("Enjoy the weekend!");  
    break;  
}
```

JavaScript Loops

1. `for` - loops through a block of code a number of times
2. `for/in` - loops through the properties of an object
3. `for/of` - loops through the values of an iterable object
4. `while` - loops through a block of code while a specified condition is true
5. `do/while` - also loops through a block of code while a specified condition is true



for Loops:

```
for (initialization; condition; after change) {  
  body  
}
```

```
for ( let i = 0; i < 10; i++ ) {  
  console.log(i)  
}
```

```
// output:  
0=BMW  
1=Volvo  
2=Saab  
3=Ford
```

for in Loops:

```
for (variable in object) {  
  body  
} // x means Index
```

```
const person = {  
  fname:"John", lname:"Doe", age:25  
};
```

```
for (let x in person) {  
  console.log(x,":",person[x])  
}
```

```
// output:-  
fname : John  
lname : Doe  
age : 25
```

for of Loops:

```
for (variable of object) {  
  // body  
}
```

```
const cars = ["BMW", "Volvo", "Saab", "Ford"];
```

```
for (let x of cars) {  
  console.log(x)  
}
```

```
// Output:  
BMW  
Volvo  
Saab  
Ford
```

while Loops:

```
while (condition) {  
  // body  
}
```

```
let i = 0;  
while (i < 5) {  
  console.log(i);  
  i++;  
}
```

```
// output:-  
0  
1  
2  
3  
4
```


do while Loops:

```
do {  
  // body  
}  
while (condition);
```

```
let i = 0;  
do {  
  console.log(i);  
  i++;  
}  
while (i < 5);  
// output:-
```

0
1
2
3
4

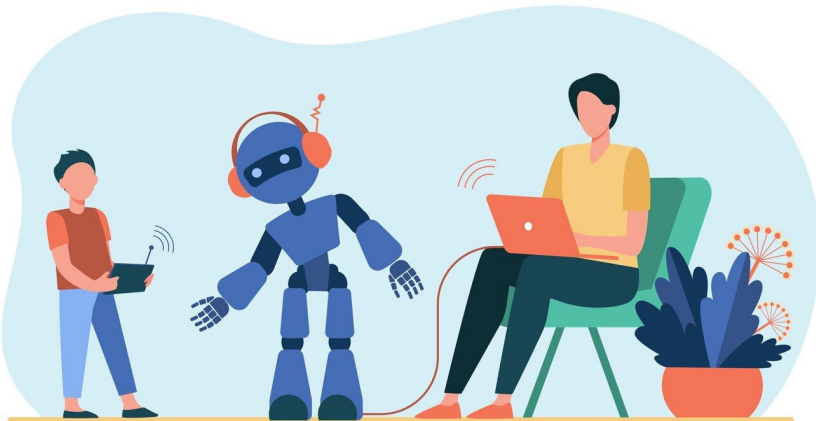
Objects (Creating new Objects)

```
// 1. Object Literals
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
};
```

```
// 2. Object Constructor
(Rarely Used)
let product = new
Object({
  name: "T-Shirt",
  price: 19.99,
});
```

```
// 3. Object's function Constructor
function Person(first, last, age, eye)
{
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}
```

```
// Create a Person object
const myFather = new
Person("John", "Doe", 50, "blue");
```



Objects

Accessing JavaScript Properties

The syntax for accessing the property of an object is:

```
objectName.property    // person.age
```

or

```
objectName["property"] // person["age"]
```

or

```
objectName[expression] // x = "age"; person[x]
```

We can access nested objects as well

like. Person.male.age;

Accessing Objects

```
let person = {  
  firstName: "John",  
  age: 30,  
};  
  
console.log(person.firstName);  
console.log(person["firstName"]);  
// Outputs: John
```

Objects

Updating JavaScript Properties

The syntax for accessing the property of an object is:

```
objectName.property = "Vijay"
```

```
// person.age
```

or

```
objectName["property"] = "Vijay"
```

```
// person["age"]
```

or

```
objectName[expression] = "Vijay"
```

```
// x = "age"; person[x]
```

We can access nested objects as well

like. `Person.male.age = "Vijay";`

Updating Objects (or adding new Properties)

```
let person = {  
  firstName: "John",  
};
```

```
console.log(person.firstName);  
// Outputs: John
```

```
person.firstName = "Vijay";  
console.log(person.firstName);  
// Outputs: Vijay
```

Objects

- The delete operator attempts to remove the specified property from the object.
- It returns true if the property was deleted successfully, and false otherwise
- (e.g., if the property doesn't exist or can't be deleted due to configuration).

```
// deleting Object property
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

// Delete the age property
delete person.age;

console.log(person);
// { firstName: "John", lastName: "Doe" }
```

Object Notes

- 1) Object can be created using an object literal, with the function constructors, or using `new Object()`
- 2) Objects are mutable: They are addressed by reference, not by value.
 - E.g. If person is an object, the following statement will not create a copy of person:
 - `const x = person; // Will not create a copy of person.`
 - The object x is not a copy of person. It is person. Both x and person are the same object.

```
const person = {  
    fname:"vijay",  
    lname:"kumar",  
    age:40,  
  
    names:{  
        firstName: "John",  
        lastName: "Doe",  
        age2: 30,  
    }  
}  
  
// Destructuring  
const {fname, ...rest} = person;  
const {fname:name, ...rest} = person;  
const {names:{age2}} = person
```


Objects Freezing / Sealing Notes

- 1) Once an Object is freezed. The Object can not be modified and in Strict Mode, it will give type error.
- 2) Once an Object is Sealed, new properties cannot be added, existing properties cannot be removed . In Strict Mode, but existing properties can be modified.

```
const obj = {  
  prop: 42,  
};  
  
Object.freeze(obj);  
  
obj.prop = 33;  
// Throws an error in strict mode  
  
console.log(obj.prop);  
// Expected output: 42
```

3. Once an object is sealed or freezed it can not be unsealed or unfreezed. we have to create a new object with same or modified properties.

```
const object1 = {  
  property1: 42,  
};  
  
Object.seal(object1);  
object1.property1 = 33;  
console.log(object1.property1);  
// Expected output: 33  
  
delete object1.property1;  
// Cannot delete when sealed  
console.log(object1.property1);  
// Expected output: 33
```

```
//Constructors (Rarely Used)
```

```
const names = new Array("Vijay","Ajay","Karan");
```

```
//Access or Update
```

```
Console.log(names[0]);  
names[0] = "Ajay";
```

```
//without constructor
```

```
const names = ["Vijay","Ajay","Karan"];
```

```
or
```

```
const names = [];
```

```
names[0]= "Vijay";
```

```
names[1]= "Ajay";
```

```
names[2]= "Karan";
```

```
// To String() method (Array to String)
```

```
Console.log(names.toString());
```

```
// String to Array
```

```
const str = "abc, def,jkl,pqr,stu";
```

```
str.split(","); // output ["abc", "def", "jkl", "pqr", "stu"];
```

```
//Adding at last index
```

```
const added = names.push("DATA");
```

```
// output index of added item
```

```
// Array Splice method
```

```
const removed = names.splice(1,1); // ["Ajay"]
```

```
names.splice(1,0,"Sanjay","panjay"); // Adding
```

```
// This method returns the deleted item in array
```

```
//remove at last index
```

```
const removed = names.pop();
```

```
// output value of removed item
```

```
const names = ["Vijay","ajay","karan"]
```

```
//Find length of array  
names.length;
```

```
//desctructuring  
const [ceo,manager,founder] = names;  
const [ceo,,founder] = names;  
const [ceo,...others] = names;
```

```
//Array Concatenation
```

```
const arr1 = ["Cecilie", "Lone"];  
const arr2 = ["Emil", "Tobias", "Linus"];  
const arr3 = ["Robin"];  
const children = arr1.concat(arr2, arr3);
```

```
// fill method
```

```
const arr = [1,2,3];  
const myArr = arr.fill(2);    // output [2,2,2]
```

```
// array from method
```

```
const myArr = Array.from("ABCDEF"); // output [A,B,C,D,E,F]  
const myArr = Array.from({length:4}); // output [,,,]  
const myArr = Array.from({length:4},function(e){return 1}); //  
output [1,1,1,1]  
const myArr = Array.from({length:4},(e,i)=>i); // output [1,2,3,4]  
const myArr = Array.from({length:4}).fill(1); // output [1,1,1,1]  
Array.from({length:26},(e,i)=>String.fromCharCode(i+65));
```

```
//typeof vs Array.isArray()
```

```
typeof names;  
Array.isArray(names);  
fruits instanceof Array;
```

```
// Define an array of objects
```

```
const people = [  
  { name: 'John', age: 30 },  
  { name: 'Alice', age: 25 },  
  { name: 'Bob', age: 35 },  
];
```

```
// Check if all people in the array are above 18 years
```

```
old
```

```
const allAdults = people.every((person) =>  
  person.age > 18);  
console.log(allAdults); // Output: true
```

```
// Array includes method
```

```
names.includes("Vijay"); // Output: true / false
```

```
fruits.includes("Vijay", 3); // Output: true / false
```

```
const index = names.indexOf("Vijay");
```

```
// Output: 4 / -1
```

```
// Array lastIndexOf method
```

```
const index = names.lastIndexOf("Vijay"); //
```

```
Output: 4 / -1
```

```
const index = names.lastIndexOf("Vijay", 3);
```

```
// Output: 6 / -1
```

Arrays Find Method

// Array lastIndexOf method

```
const index = names.lastIndexOf("Vijay"); // Output: 4 / -1
```

```
const index = names.lastIndexOf("Vijay", 3); // Output: 6 / -1
```

// Array find method

```
const ages = [3, 19, 18, 20];
```

```
const value = ages.find((e)=>e > 18); // 19
```

// Array findIndex method

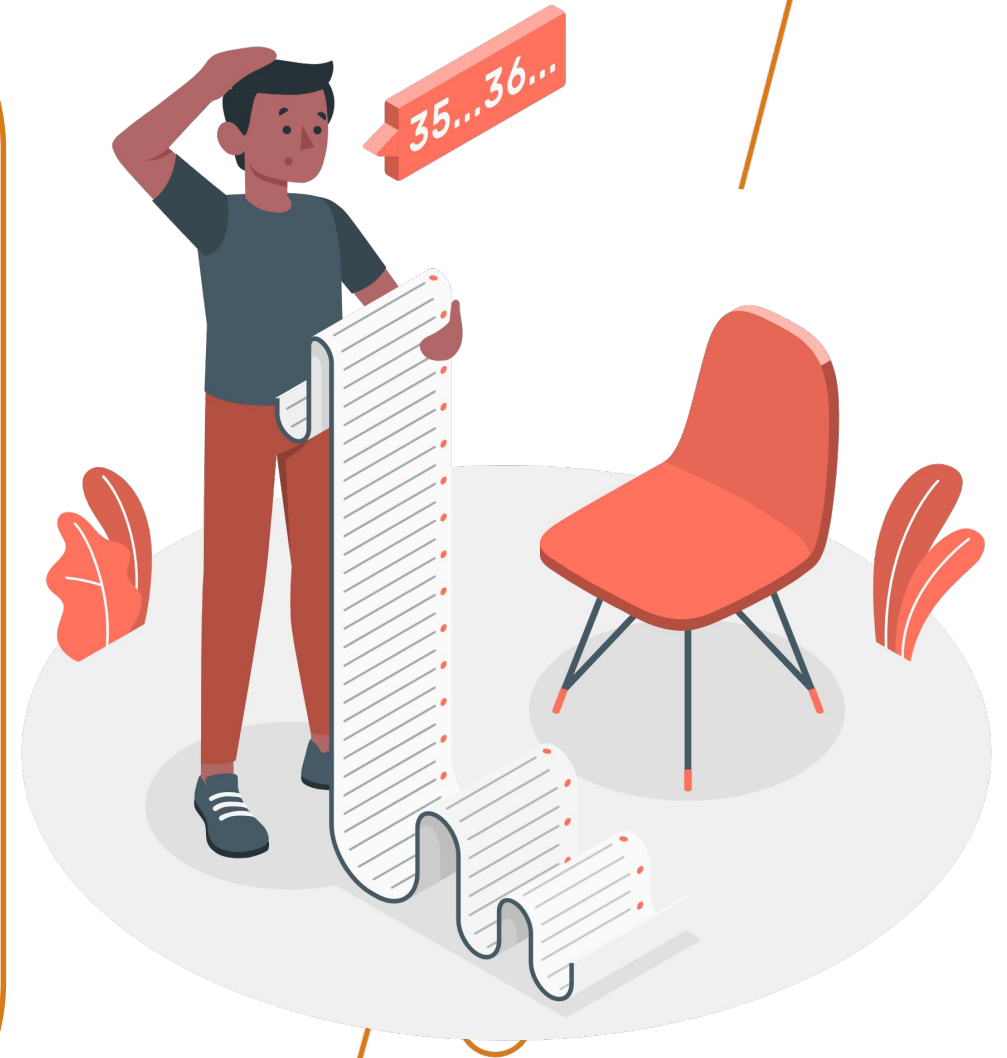
```
const value = ages.findIndex((e)=>e > 18); // 1
```

// Array findLast method

```
const value = ages.findLast((e)=>e > 18); // 20
```

// Array findLastIndex method

```
const value = ages.findLastIndex((e)=>e > 18); // 3
```



Array Sorting Methods



```
// Array Sort method (Original Array)
const names = ["B","A","F","D","E","F"];
names.sort();
// Output: ['A', 'B', 'D', 'E', 'F', 'F']
names.reverse();
// Output: ['F', 'F', 'E', 'D', 'B', 'A']
```

```
//Sorting Numbers Ascending
ages.sort((a,b)=>a-b);
// Output: [3, 18, 19, 20]
ages.sort((a,b)=>b-a);
// Output: [20, 19, 18, 3]
```

```
const names = ["B", "A", "F", "D", "E", "c", "a"];
names.sort((a, b) =>
a.toLowerCase().localeCompare(b.toLowerCase()));
console.log(names);
// Output: ["a", "A", "B", "c", "D", "E", "F"]
```

Arrays Filter Methods

\\ Syntax:

```
function filterFunction(value, index, array){  
}  
array.filter(filterFunction);
```

```
function isBigEnough(value) {  
  return value >= 10;  
}
```

```
const filtered = [12, 5, 8, 130, 44].filter(isBigEnough);  
console.log(filtered);  
// Expected Output: [12, 130, 44]
```

```
const words = ['spray', 'elite', 'exuberant',  
  'destruction', 'present'];  
const result = words.filter((word) => word.length  
  > 6);  
console.log(result);
```

```
// Expected output: Array ["exuberant",  
  "destruction", "present"]
```


forEach and map method of array

1. **forEach** method doesn't return new array. **forEach** method is faster than **map**.
2. **map** method return new array. **map** method is slower than **forEach**.

```
const numbers = [1, 2, 3, 4];
```

```
//forEach method (DOESN'T RETURN ANYTHING)
```

```
numbers.forEach(number => console.log(number * 2));
```

```
// Output: 2, 4, 6 , 8
```

```
//forEach method (RETURN NEW ARRAY)
```

```
const doubledNumbers = numbers.map(number =>  
number * 2);
```

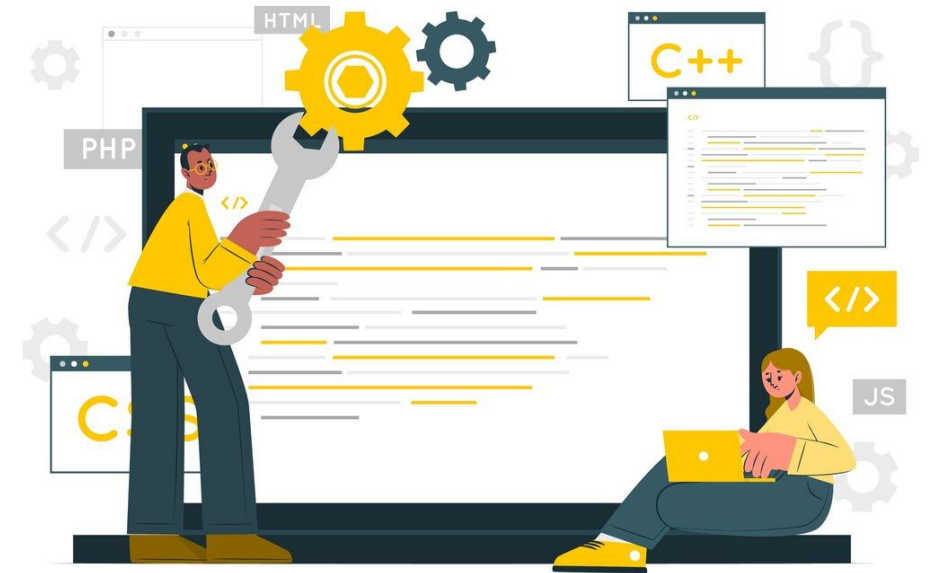
```
// doubledNumbers = [2, 4, 6, 8] (creates a new array)
```

JavaScript Functions

function is a reusable blocks of code that perform a specific task. They encapsulate a set of instructions that can be executed when invoked (called upon) with or without arguments (inputs) and may or may not return something.

Four Types of functions

1. returning and Argumented
2. returning and non Argumented
3. non returning and non Argumented
4. non returning and Argumented



//Type 1: non returning and non Argumented

// step 1: define function

```
function functionName(){  
  console.log("Hi! Vijay.");  
}
```

// step 2: call/invoke the function

```
functionName();
```

// output : HI! Vijay.

1. you can add as many arguments with any name at the time of creating function.
2. when we pass the value of arguments at the time of calling function the values are called parameters.

// Type 2: non returning and Argumented

// step 1: define function. arguments can be //
considered as variables. we will pass their //
value at the time of calling function

```
function functionName(argument1, arg2){  
  console.log("Hi! ", argument1, arg2);  
}
```

// step 2: call/invoke the function with argument
values (also called parameter).

```
functionName("Vijay","Kumar");
```

// output : HI! Vijay Kumar.

// Type 3: Returning and Argumented
// step 1: define function

```
function functionName(argument1, arg2){  
    const result = argument1 + arg2;  
    return result;  
}
```

// step 2: call/invoke the function and store return value in new variable.

```
const data = functionName(9,7);  
console.log(data);  
// output : 16
```

// Type 4: Returning and non Argumented
// step 1: define function

```
function functionName(){  
    const result = 84 + 5;  
    return result;  
}
```

// step 2: call/invoke the function and store return value in new variable.

```
const data = functionName();  
console.log(data);  
// output : 89
```

Both functions are same they give same result with same arguments.

// Using function Keyword

```
function functionName(argument1, arg2){  
  const result = argument1 + arg2;  
  return result;  
}
```

// step 2: call/invoke the function

```
const data = functionName(9,7);  
console.log(data);  
// output : 16
```

// Using arrow function

```
const functionName = (argument1, arg2) => {  
  const result = argument1 + arg2;  
  return result;  
}
```

// step 2: call/invoke the function

```
const data = functionName(9,7);  
console.log(data);  
// output : 16
```

```
const functionName = function(argument1, arg2){  
  const result = argument1 + arg2;  
  return result;  
}
```

// Anonymous Functions

```
function(argument1, arg2){  
  const result = argument1 + arg2;  
  return result;  
}
```

// Anonymous Functions. we pass this Anonymous function as parameter to methods. e.g.
array.filter(anonymous function)

// Self Executing Anonymous function

```
(function() {  
  const result = 10 + 6;  
  console.log(result);  
})();
```

// output : 16

Learn About: Global and Local Scope of variables.

Recursive function means a function calling itself inside the body of function. e.g. factorial of a function (see right side block).

```
// Nested functions
function outside() {
  const x = 5;
  function inside(x) {
    return x * 2;
  }
  return inside;
}
```

```
outside(5)(2); // output: 10
```

```
function factorial(n) {
  if (n === 0 || n === 1) {
    return 1;
  } else {
    return n * factorial(n - 1);
  }
}

console.log(factorial(2)); // 2
console.log(factorial(3)); // 6
console.log(factorial(4)); // 24
```


Math Basic Methods

1. `abs(x)`
2. `sqrt(x)`
3. `cbrt(x)`
4. `ceil(x)`
5. `floor(x)`
6. `round(x)`
7. `max(x1, x2, x3, ...xn)`
8. `min(x1, x2, x3, ...xn)`
9. `pow(x, y)`
10. `random(no_argument)`

[Go To Mozilla's Math Reference](#)

String Methods - 1

- 1.charAt(number)
- 2.charCodeAt(number) => get UTF-16 code
- 3.concat(string)
- 4.endsWith(string)
- 5.String.fromCharCode(number) => get Character from UTF-16
- 6.includes(string, start_index:number)
- 7.indexOf(string, start_index:number)
- 8.lastIndexOf(string, start_index:number)
- 9.length => property
- 10.localeCompare(string)
- 11.match(string)
- 12.padEnd(index, string)
- 13.padStart(index, string)
- 14.repeat(count)
- 15.replace(searchValue, newValue)
- 16.replaceAll(searchValue, newValue)
- 17.search(searchValue)
- 18.slice(start:number, end:number)

[Go To Mozilla's String Reference](#)

String Methods - 2

20. `split(seperator, limit)`
21. `startsWith(searchValue, start)`
23. `substring(start, end)`
24. `toLowerCase()`
25. `toUpperCase()`
26. `trim()`
27. `trimEnd()`
28. `trimStart()`

Date Methods

1. `new Date(date: optional)` => give Date in Date Object
2. `Date.now()` => give date in Milliseconds Format
3. `date.toLocaleString()`
4. `date.toLocaleDateString()`
5. `date.toLocaleTimeString()`

date setter and getter methods

1. `date.getTime()` / `date.setTime()`
2. `getDate()` / `setDate()`
3. `getMonth` / `setMonth()`
4. `getDay()`
5. `getFullYear()` / `setFullYear()`
6. `getHours()` / `setHours()`
7. `getMilliseconds()` / `setMilliseconds()`
8. `getMinutes()` / `setMinutes()`
9. `getSeconds()` / `setSeconds()`

try...catch...finally

```
try {  
    // your code here  
} catch (e) {  
    console.error(e);  
} finally {  
    // finally is optional block.  
}
```

throw:

it is used to throw an error, intentionally

```
try {  
    if(x == "") throw "empty";  
    if(isNaN(x)) throw "not a number";  
    x = Number(x);  
    if(x < 5) throw "too low";  
    if(x > 10) throw "too high";  
}  
catch(err) {  
    console.log("Input is " + err);  
}
```

fetch api :

```
fetch("url")  
  .then( x => x.json())  
  .then( e => console.log(e) )
```

note: always use fetch api in try catch block and using fetch api

```
try {  
  fetch("https://api_url")  
  .then((x) => x.json())  
  .then((e) => console.log(e));  
} catch (error) {  
  console.error(error);  
} finally {  
  console.log("Program Executed!")  
}
```

async...await

async

it use to alternative default behaviour of a function from synchronous to asynchronous.

await:

it is used to hold the calls and pause the function till it gets the result of that call.

```
async function functionName() {  
  try {  
    const x = await fetch("https://api_url");  
    const y = await x.json();  
    console.log(y);  
  } catch (error) {  
    console.error(error);  
  } finally {  
    console.log("Program Executed!");  
  }  
}
```

```
functionName();
```

Javascript Regex - 1

1. What are Regular Expressions?

- Patterns used to match specific characters or sequences of characters in strings.
- Powerful tool for text searching, extraction, and validation.

2. Syntax:

- Regular expressions are written within forward slashes (`/pattern/`).
- Can contain literal characters, special characters (metacharacters), and quantifiers.

3. Literal Characters:

- Match themselves exactly.
- Examples: `a`, `b`, `c`, `1`, `2`, `3`.

4. Special Characters (Metacharacters):

Have special meanings within regular expressions.

Common examples:

- `.` - Matches any single character except newline.
- `^` - Matches the beginning of the string.
- `$` - Matches the end of the string.
- `[]` - Character class: Matches one character from the set inside the brackets.
- `*` - Quantifier: Matches the preceding character zero or more times.
- `+` - Quantifier: Matches the preceding character one or more times.
- `?` - Quantifier: Matches the preceding character zero or one time.
- `|` - OR operator: Matches one of the alternatives separated by the pipe.

Javascript Regex - 2

- `\d` - Matches a digit (0-9).
- `\w` - Matches a word character (alphanumeric or underscore).
- `\s` - Matches a whitespace character (space, tab, newline).

5. Quantifiers:

- Specify how many times the preceding character or group can be matched.
- Examples:
 - `*` - Zero or more times.
 - `+` - One or more times.
 - `?` - Zero or one time.
 - `{n}` - Exactly n times.
 - `{n, m}` - At least n but no more than m times.

6. Escaping Characters:

- To match a special character literally, escape it with a backslash (`\`).
- Example: `\.` matches a literal dot (`.`).

7. Grouping:

- Use parentheses `()` to group characters or expressions.
- Treated as a single unit for quantifiers or other operations.

Javascript Regex - 3

8. Flags (Modifiers):

- Optional flags added after the closing slash to modify the behavior of the regular expression.
- Common flags:
 - **i** - Case-insensitive matching.
 - **g** - Global search: Matches all occurrences in the string.
 - **m** - Multiline mode: Recognizes **^** and **\$** at the beginning/end of each line.

9. Methods for Using RegEx:

- **test(str)** - Returns true if the pattern is found in the string. e.g. "Vijay".test(/^a/)
- **exec(str)** - Returns an object with information about the first match, or null if no match.
- **match(str)** - Returns an array of all matches in the string (with **g** flag) or the first match (without **g**).
- **replace(str, replacement)** - Replaces all matches in the string with the replacement string.
- **search(str)** - Returns the index of the first match, or -1 if no match.

10. Examples:

- Match a phone number: **/^\d{3}-\d{3}-\d{4}\$/g** (with flags **g** for global match)
- Extract email usernames: **/^(\w+)\@/** (capture username in parentheses)
- Validate passwords: **/^(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}\$/** (complex password with at least 8 characters, a digit, a lowercase letter, and an uppercase letter)